

BookTrack

Memoria completa del proyecto de fin de ciclo

CFGS Desarrollo de Aplicaciones Web
Curso 2025/2026

Álvaro Bravo Marcos
Juan Rodríguez Lacoste
Hugo Gago Vicente

Colegio Calasanz Salamanca

Índice

1. Introducción
2. Descripción de la aplicación
3. Planificación del proyecto
4. Análisis del sector
5. Diseño del proyecto
6. Planificación y ejecución
7. Evaluación y control de calidad
8. Conclusiones y referencias

1. Introducción

La aplicación propuesta lleva por nombre BookTrack. El proyecto consiste en desarrollar una plataforma web que permita a los usuarios gestionar y realizar un seguimiento de sus lecturas, registrar libros terminados, en progreso y pendientes, así como acceder a estadísticas personalizadas. Con esta aplicación se pretende facilitar la organización personal de lecturas y fomentar hábitos lectores mediante una interfaz intuitiva.

2. Descripción de la aplicación

La aplicación BookTrack ofrecerá una serie de funcionalidades orientadas a la gestión de la actividad lectora. El usuario podrá registrarse e iniciar sesión para acceder a su espacio personal. Dentro de la plataforma encontrará varios apartados:

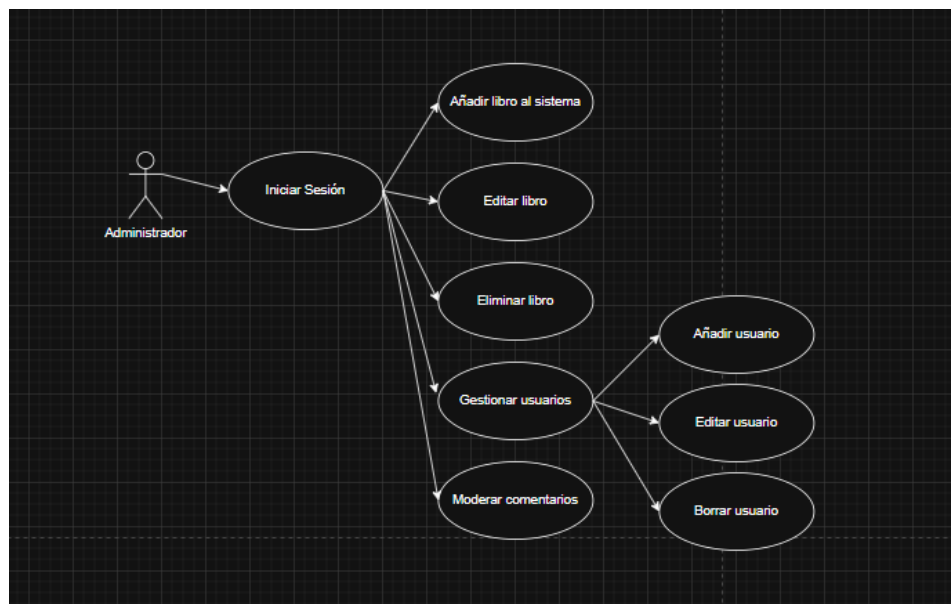
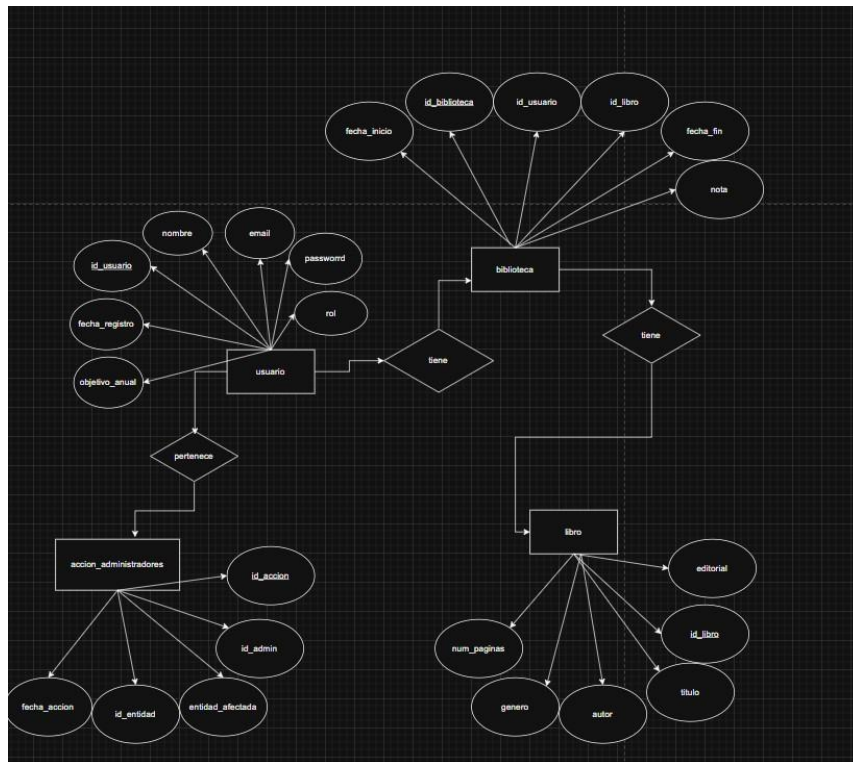
- Resumen personal: resumen del estado de sus lecturas, número de libros leídos en el mes actual y en el año, progreso de lecturas en curso, estadísticas y progreso mensual.
- Biblioteca: sección donde el usuario podrá añadir libros, clasificarlos como leído, pendiente o en curso, y modificar su estado durante el seguimiento.
- Detalle del libro: ficha específica con título, autor, género, editorial, número de páginas, nota personal y comentarios.
- Buscador: localización de libros dentro de la biblioteca mediante filtros avanzados por texto, estado, autor, género y editorial.
- Estadísticas: generación de gráficas sobre libros leídos por mes, distribución por géneros, número de libros por periodo y progreso del objetivo anual definido por el usuario.

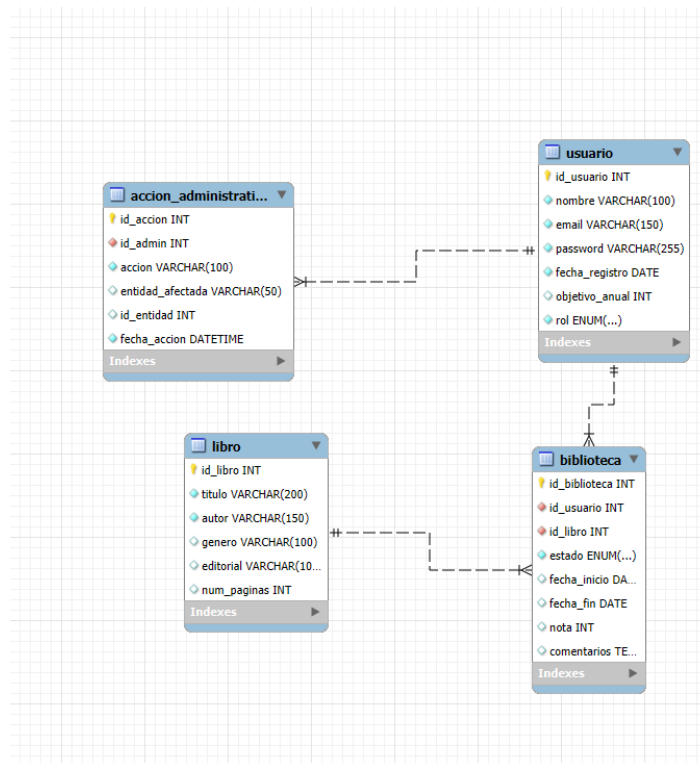
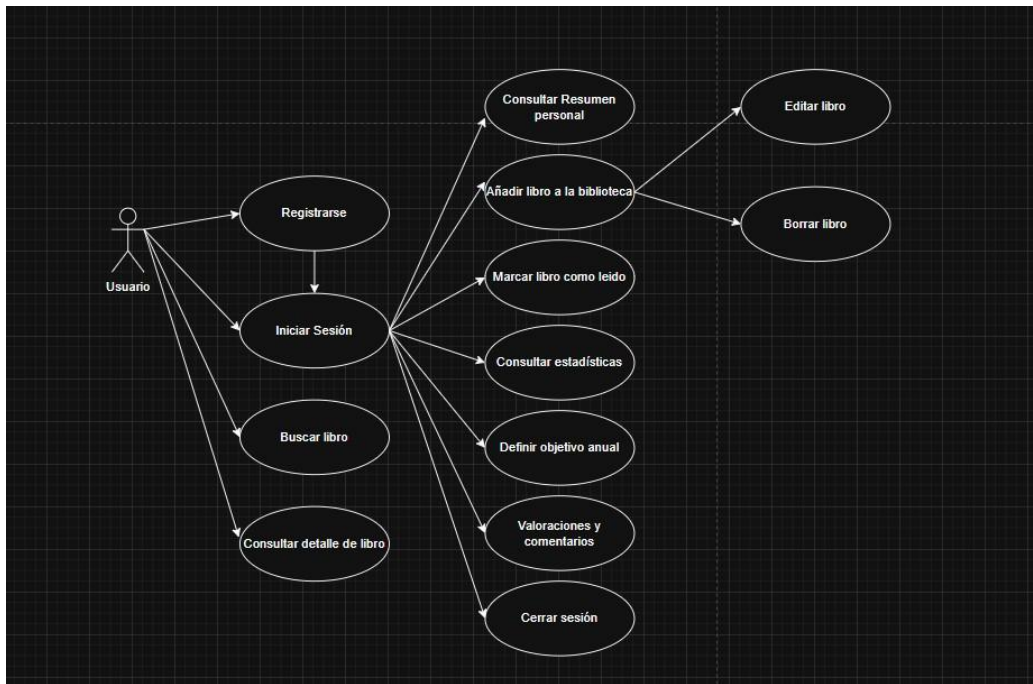
El flujo de navegación será sencillo: tras iniciar sesión, el usuario accederá al resumen; desde ahí podrá moverse entre la biblioteca, las estadísticas, el catálogo y su perfil. Las pantallas estarán diseñadas para ser claras y facilitar el acceso rápido a las diferentes funcionalidades. Estos requisitos constituyen el análisis previo del proyecto y sirven como base para el diseño posterior.

3. Planificación del proyecto

La planificación estimada del proyecto es la siguiente:

- Semana 1-2: definición de requisitos, prototipado de interfaces y preparación de la memoria inicial.
- Semana 3-5: desarrollo del backend, API, base de datos y modelos principales.
- Semana 6-8: implementación del frontend y conexión con el backend.
- Semana 9-10: desarrollo del módulo de estadísticas y optimización general.
- Semana 11: pruebas, correcciones y documentación técnica.
- Semana 12: preparación de la memoria final y entrega del proyecto.





4. Análisis del sector

4.1 Clasificación de empresas del sector

El proyecto se sitúa en el sector del desarrollo de software, concretamente en aplicaciones web de productividad personal, analítica de datos y gestión de contenidos. En este sector pueden distinguirse empresas de producto propio SaaS, consultoras de desarrollo a medida, agencias digitales, startups tecnológicas, editoriales con plataformas digitales y proveedores de infraestructura cloud. Las primeras comercializan una aplicación recurrente; las segundas prestan servicios por proyecto; las agencias combinan diseño, experiencia de usuario y marketing; las startups priorizan escalabilidad y captación de usuarios; y los proveedores cloud ofrecen alojamiento, bases de datos y automatización de despliegues. BookTrack se aproxima a un producto

SaaS vertical de nicho, orientado al seguimiento personal de lecturas y con posibilidad de evolución hacia comunidad lectora, recomendaciones o integración con catálogos externos.

4.2 Empresa tipo: estructura organizativa y funciones

La empresa tipo para este proyecto sería una pyme tecnológica o startup de software con estructura funcional ligera. Dirección define estrategia, modelo de negocio y alianzas; Producto prioriza requisitos, experiencia de usuario y hoja de ruta; Desarrollo implementa backend, frontend y base de datos; QA verifica requisitos, seguridad y rendimiento; Sistemas o DevOps gestiona contenedores, entornos y despliegue; Marketing y ventas capta usuarios y comunica valor; Atención al cliente recoge incidencias y propuestas; y Administración controla facturación, fiscalidad y protección de datos. En fases iniciales varias funciones pueden recaer en una misma persona, pero la separación de responsabilidades permite mantener calidad y trazabilidad.

4.3 Necesidades más demandadas

Las empresas del sector reciben una demanda creciente de aplicaciones accesibles desde navegador, seguras, responsive, fáciles de mantener y con despliegue reproducible. Los clientes solicitan autenticación robusta, aislamiento de datos por usuario, paneles de estadísticas, automatización de procesos, bajo coste de infraestructura, capacidad de escalar y documentación técnica suficiente para mantener el producto. En BookTrack estas necesidades se traducen en registro e inicio de sesión con JWT, CRUD de libros, filtros, dashboard, estadísticas y persistencia en MySQL, manteniendo arquitectura por capas para facilitar futuras ampliaciones.

4.4 Oportunidades de negocio previsible

El crecimiento del consumo de servicios digitales y la normalización de modelos SaaS generan oportunidades para productos especializados de bajo coste. En el ámbito lector existen oportunidades en clubes de lectura, centros educativos, bibliotecas, usuarios particulares con objetivos anuales y creadores de contenido literario. BookTrack puede evolucionar hacia planes freemium, funciones premium, exportación de datos, integración con APIs bibliográficas, recomendaciones, rankings o uso en centros educativos. La oportunidad principal es ofrecer una alternativa sencilla y privada frente a plataformas generalistas, centrada en seguimiento personal, analítica clara y propiedad de datos por usuario.

4.5 Tipo de proyecto requerido

La demanda identificada requiere un proyecto de desarrollo de aplicación web full-stack con arquitectura cliente-servidor, API REST securizada, persistencia relacional y despliegue contenedorizado. No se trata solo de una página informativa, sino de un sistema transaccional con usuarios, sesiones, operaciones CRUD, reglas de negocio, estadísticas calculadas y controles de seguridad. Por ello se adopta Java 17 con Spring Boot para el backend, Vue 3 con Vite para el frontend, MySQL 8.0 para la persistencia y Docker Compose para reproducir el entorno completo.

4.6 Características específicas del proyecto

BookTrack debe ofrecer autenticación stateless con JWT de 24 horas, contraseñas cifradas con BCrypt, aislamiento total de datos por usuario, validación de estados de lectura, fechas automáticas al pasar de pendiente a en curso y a leído, filtros combinables y dashboard de progreso anual. A nivel técnico requiere separación Controller-Service-Repository, entidades JPA, repositorios JpaRepository, configuración de seguridad con JwtTokenProvider, JwtAuthenticationFilter y SecurityConfig, además de contenedores independientes para base de datos, backend y frontend. La solución debe poder ejecutarse de forma local y desplegarse en infraestructura cloud con cambios mínimos de configuración.

4.7 Obligaciones fiscales, laborales y prevención de riesgos

En España, una empresa que comercialice o desarrolle software debe atender obligaciones fiscales como alta censal, IAE cuando proceda, IVA, IRPF si actúa como autónomo, Impuesto sobre Sociedades si opera como sociedad, retenciones e informativas. En el ámbito laboral debe cumplir el Estatuto de los Trabajadores, contratación, cotización a la Seguridad Social, registro horario, igualdad y normativa aplicable a teletrabajo si lo hubiera. En prevención de riesgos laborales debe evaluar riesgos de oficina y uso de pantallas, ergonomía, pausas, iluminación, carga mental y ciberseguridad operativa. Además, el tratamiento de usuarios, correos y contraseñas exige cumplir RGPD y LOPDGDD, aplicando minimización, confidencialidad, información al usuario, base jurídica y medidas técnicas adecuadas.

4.8 Ayudas y subvenciones

Para incorporar nuevas tecnologías existen programas públicos aprovechables por pymes y autónomos, como Kit Digital para soluciones de digitalización, ayudas de Red.es y oficinas Acelera pyme, programas CDTI para I+D e innovación empresarial, ENISA para financiación de emprendimiento innovador y convocatorias autonómicas o locales relacionadas con transformación digital. Para BookTrack podrían financiarse servicios de presencia web avanzada, ciberseguridad, analítica, cloud, consultoría tecnológica o evolución del prototipo a producto comercial.

4.9 Guión de trabajo

El guión de trabajo se organiza en: análisis del problema y del mercado, definición de requisitos funcionales y no funcionales, diseño del modelo de datos, diseño de arquitectura, planificación temporal, implementación del backend, implementación del frontend, integración con MySQL, despliegue con Docker Compose, pruebas funcionales y de seguridad, elaboración de documentación, evaluación de calidad y preparación de la entrega final. Este orden evita desarrollar interfaces sin reglas de negocio definidas y permite validar progresivamente cada capa del sistema.

5. Diseño del proyecto

5.1 Recopilación de información y alcance

La información de partida procede del anteproyecto, de los requisitos funcionales definidos para BookTrack y de las tecnologías seleccionadas. Se han identificado usuarios finales que desean registrar lecturas, consultar progreso y filtrar su biblioteca; también se han considerado requisitos técnicos de seguridad, persistencia, mantenibilidad y despliegue. El alcance incluye una aplicación web funcional con autenticación, gestión de libros, catálogo global, dashboard, estadísticas y objetivo anual configurable. Quedan fuera del alcance inicial pagos, recomendaciones basadas en inteligencia artificial, integración con ISBN externo y aplicación móvil nativa, aunque se contemplan como posibles mejoras futuras.

5.2 Viabilidad técnica

La viabilidad técnica es alta porque todas las tecnologías son maduras, documentadas y compatibles entre sí. Java 17 y Spring Boot 3.4.0 permiten crear APIs REST seguras; Spring Security 6 y JWT 0.12.6 cubren autenticación stateless; Vue 3 con Vite 5 facilita un frontend modular; Axios simplifica llamadas HTTP; MySQL 8.0 se integra con Spring Data JPA e Hibernate; Maven automatiza compilación y dependencias; Docker Compose reproduce el entorno con tres servicios. El uso de `ddl-auto=update` resulta adecuado para desarrollo y prototipo académico, aunque en producción se recomendaría migrar a herramientas como Flyway o Liquibase para controlar versiones de esquema.

Componente	Tecnología	Justificación
Backend	Java 17, Spring Boot 3.4.0	API REST robusta, inyección de dependencias, ecosistema de seguridad y persistencia.
Seguridad	Spring Security 6, JWT, BCrypt	Sesiones stateless, protección de endpoints y almacenamiento seguro de contraseñas.
Frontend	Vue 3, Vite 5, Axios	Interfaz reactiva, compilación rápida y comunicación sencilla con la API.
Persistencia	MySQL 8.0, JPA/Hibernate	Modelo relacional, transacciones y generación inicial de esquema.
Despliegue	Docker, Docker Compose	Entorno reproducible con red interna, volumen persistente y dependencias entre servicios.

5.3 Fases del proyecto y objetivos

Las fases se dividen en análisis, diseño, construcción, integración, pruebas, despliegue y documentación. El objetivo general es desarrollar una aplicación web que permita al usuario controlar su actividad lectora de forma segura y organizada. Como objetivos específicos se establecen: registrar usuarios, autenticar con JWT, gestionar libros, cambiar estados de lectura, aplicar filtros, mostrar un resumen de progreso, generar estadísticas mensuales y por género, configurar objetivo anual, ofrecer catálogo global y garantizar aislamiento de datos por usuario. El alcance se limita a un prototipo funcional completo para entorno académico, con capacidad de ser desplegado mediante contenedores.

5.4 Recursos materiales y personales

Recurso	Descripción	Uso previsto
Equipo de desarrollo	Ordenador con JDK 17, Node.js, Maven, Docker Desktop o motor Docker	Programación, pruebas locales y construcción de imágenes.
IDE y control de versiones	IntelliJ, Eclipse o VS Code, Git y repositorio remoto	Edición de código, historial de cambios y colaboración.
Base de datos	MySQL 8.0 Community en contenedor	Persistencia de usuarios y libros.
Personal	1-3 desarrolladores DAW con roles rotativos	Backend, frontend, pruebas, documentación y despliegue.
Infraestructura	Servidor VPS o proveedor cloud con soporte Docker	Despliegue demostrable y acceso externo.

5.5 Presupuesto económico e identificación de financiación

El presupuesto distingue costes de desarrollo académico, herramientas y despliegue. Se consideran licencias gratuitas o community cuando procede, y un coste estimado de hosting si se quisiera publicar el prototipo. Docker Desktop puede tener uso gratuito en escenarios personales o educativos, pero para uso empresarial debe revisarse su licencia; alternatively puede usarse Docker Engine en Linux. MySQL Community no implica coste de licencia.

Concepto	Unidades/Horas	Coste unitario	Subtotal
Análisis y diseño funcional	25 h	20 €/h	500 €

Concepto	Unidades/Horas	Coste unitario	Subtotal
Desarrollo backend	55 h	20 €/h	1.100 €
Desarrollo frontend	55 h	20 €/h	1.100 €
Pruebas, seguridad y correcciones	25 h	20 €/h	500 €
Documentación y memoria	20 h	20 €/h	400 €
MySQL 8.0 Community	1	0 €	0 €
Java, Maven, Vue, Vite, Axios	1	0 €	0 €
Docker / Docker Compose	1	0 € académico	0 €
Hosting de contenedores VPS	3 meses	12 €/mes	36 €
Dominio opcional	1 año	12 €/año	12 €
TOTAL ESTIMADO			3.648 €

Las necesidades de financiación son reducidas en fase de prototipo: basta con equipo propio y servicios gratuitos o de bajo coste. Para una versión comercial se necesitaría financiación para infraestructura, protección legal, soporte, marketing, monitorización y mantenimiento. Podría cubrirse mediante recursos propios, ayudas públicas de digitalización, incubadoras o financiación para proyectos innovadores.

5.6 Documentación de diseño y control de calidad

La documentación necesaria incluye especificación de requisitos, casos de uso, diagrama de arquitectura, modelo entidad-relación, definición de endpoints, guía de despliegue, manual de usuario y plan de pruebas. Para garantizar calidad se controlan seguridad de autenticación, validación de entradas, aislamiento por usuario, integridad de estados, consistencia de fechas, rendimiento básico de consultas, usabilidad, compatibilidad de navegadores y reproducibilidad del despliegue.

5.7 Arquitectura y despliegue con Docker Compose

Navegador usuario

|

v

Frontend Vue 3 (contenedor frontend, puerto 3000)

| HTTP / Axios / API REST

v

Backend Spring Boot (contenedor backend, puerto 8081)

| JDBC / JPA / Hibernate

v

MySQL 8.0 (contenedor db, red interna, volumen db_data)

docker-compose.yml orquesta tres servicios. El servicio db utiliza la imagen mysql:8.0, guarda datos en el volumen persistente db_data y expone un healthcheck con mysqladmin ping. El backend se construye con Dockerfile multi-stage: una fase build basada en maven:3.9.9-eclipse-temurin-17 compila el proyecto y una fase final con eclipse-temurin:17-jre-alpine ejecuta el JAR en una imagen ligera. El backend recibe SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME, SPRING_DATASOURCE_PASSWORD y SPRING_JPA_HIBERNATE_DDL_AUTO, publica el puerto 8081 y depende de db con condición service_healthy. El frontend se construye en su propio Dockerfile, publica el puerto 3000 y depende del backend. Los contenedores se comunican por nombre de servicio dentro de la red interna de Compose, evitando direcciones IP fijas.

6. Planificación y ejecución

6.1 Secuenciación de actividades, recursos y tiempos

Fase	Actividades	Recursos	Tiempo
1. Análisis	Requisitos, alcance, entidades y casos de uso	Equipo, documentación y entrevistas simuladas	Semanas 1-2
2. Backend	Entidades, repositorios, servicios, controladores y seguridad JWT	JDK 17, Spring Boot, Maven y MySQL	Semanas 3-5
3. Frontend	Vistas, componentes, formularios, Axios y rutas	Vue 3, Vite y navegador	Semanas 6-8
4. Estadísticas	Dashboard, objetivo anual, gráficos y filtros	Backend, frontend y datos de prueba	Semanas 9-10
5. Pruebas	Pruebas funcionales, seguridad básica y correcciones	Postman, navegador, logs y Docker	Semana 11
6. Entrega	Memoria, manuales, despliegue y defensa	PDF, repositorio y demo	Semana 12

La secuencia respeta dependencias técnicas: primero se define el dominio, después se implementa persistencia y API, posteriormente se conecta la interfaz y finalmente se validan estadísticas y despliegue. La logística incluye repositorio Git compartido, ramas de trabajo, datos de prueba, entorno Docker local y copia de seguridad del código antes de cada entrega.

6.2 Permisos, autorizaciones y procedimientos de ejecución

Para un prototipo académico no se requieren licencias privativas si se usan herramientas open source y versiones community. Sí es necesario respetar licencias de dependencias, no incluir datos personales reales sin consentimiento, configurar contraseñas fuera del código fuente y cumplir las normas del centro educativo. En un despliegue público serían necesarios aviso legal, política de privacidad, política de cookies si procede, contrato con proveedor de hosting y análisis de tratamiento de datos. El procedimiento de ejecución consiste en levantar la base de datos, esperar healthcheck, arrancar backend, arrancar frontend, probar endpoints, crear usuario de prueba y validar flujos de lectura.

- Clonar repositorio y revisar variables de entorno.
- Ejecutar `docker compose up --build`.
- Verificar healthcheck de MySQL y logs del backend.
- Acceder al frontend en `http://localhost:3000`.
- Registrar usuario, iniciar sesión y probar CRUD de libros.
- Revisar dashboard, estadísticas, catálogo y objetivo anual.

6.3 Valoración económica de implementación

La implementación representa principalmente coste de horas de trabajo. En un contexto académico, el coste monetario directo es casi nulo porque las herramientas son gratuitas y el entorno puede ejecutarse en local. En un contexto empresarial, la mayor partida corresponde a desarrollo y mantenimiento, seguida de hosting, copias de seguridad, monitorización, dominio y posible soporte jurídico para protección de datos. La valoración económica estimada de 3.648 € es coherente para un MVP educativo de alcance controlado; una versión productiva con alta disponibilidad, CI/CD, analítica avanzada y soporte podría superar ampliamente esa cifra.

6.4 Plan de riesgos y prevención

Riesgo	Prob.	Impacto	Medida preventiva
Pérdida de datos durante pruebas	Media	Alto	Usar volumen persistente, copias de seguridad y datos semilla exportables.
Errores de seguridad JWT	Media	Alto	Validar firma, expiración, filtros y rutas protegidas; no exponer secretos.
Acoplamiento entre capas	Media	Medio	Mantener patrón Controller-Service-Repository y DTOs cuando sea necesario.
Retrasos por integración frontend-backend	Media	Medio	Definir endpoints antes, probar con Postman y documentar contratos JSON.
Problemas de despliegue Docker	Media	Medio	Usar healthcheck, depends_on service_healthy y variables de entorno claras.
Vulnerabilidades en dependencias	Baja	Alto	Actualizar versiones, revisar avisos Maven/npm y evitar librerías innecesarias.
Fatiga visual y postural	Media	Medio	Aplicar pausas, ergonomía, iluminación y organización de tareas.
Incumplimiento de protección de datos	Baja	Alto	Minimización, cifrado de contraseñas, privacidad por diseño y documentación legal.

6.5 Asignación de recursos y documentación de implementación

La asignación de recursos se realiza por componentes: backend para entidades, servicios, repositorios y seguridad; frontend para vistas, formularios, estado de sesión y llamadas API; base de datos para persistencia y relaciones; DevOps para Dockerfiles, Compose y variables; QA para pruebas funcionales y regresión. La documentación de implementación debe incluir estructura del repositorio, instrucciones de construcción, descripción de variables de entorno, endpoints principales, modelo de datos, comandos de arranque, usuario de prueba si procede y registro de incidencias resueltas.

6.6 Modelo de datos y endpoints implementados

Entidad/Endpoint	Contenido o función
users	id, username único, name, email único, password cifrada, annualGoal.
books	id, title, author, genre, publisher, pages, rating, comments, status, userId, startDate, endDate.
Relación	Un usuario tiene muchos libros; books.userId referencia users.id.
POST /api/auth/register - /login	Registro e inicio de sesión con JWT.
GET/POST/PUT/DELETE /api/books	Gestión CRUD de libros del usuario autenticado.
PUT /api/books/{id}/status	Cambio de estado y actualización automática de fechas.
GET /api/dashboard/summary - /statistics	Resumen y estadísticas de lectura.

Entidad/Endpoint	Contenido o función
PUT /api/dashboard/goal	Actualización del objetivo anual.
GET /api/catalog	Catálogo global con libros de todos los usuarios.

7. Evaluación y control de calidad

7.1 Procedimiento de evaluación de actividades

La evaluación se realiza al finalizar cada fase y antes de la entrega final. En análisis se comprueba que los requisitos cubran las necesidades del usuario; en diseño se valida que arquitectura y modelo de datos soporten esos requisitos; en construcción se revisa código, estructura de paquetes y cumplimiento de capas; en integración se prueba comunicación frontend-backend-base de datos; en despliegue se verifica que Docker Compose arranque los tres servicios; y en documentación se comprueba coherencia entre memoria, código y funcionalidades. Cada actividad se considera superada cuando existe evidencia verificable: commit, captura, prueba ejecutada, endpoint probado o documento actualizado.

7.2 Indicadores de calidad

Indicador	Criterio de aceptación
Funcionalidad	El usuario puede completar registro, login, CRUD, filtros, dashboard y estadísticas sin errores bloqueantes.
Seguridad	Las rutas privadas exigen JWT válido y las contraseñas se almacenan con BCrypt.
Aislamiento de datos	Un usuario no puede consultar, modificar ni borrar libros de otro usuario.
Persistencia	Los datos sobreviven al reinicio gracias al volumen db_data de MySQL.
Usabilidad	La navegación es clara y las acciones principales están disponibles en pocos pasos.
Mantenibilidad	El código respeta capas y evita lógica de negocio en controladores o componentes visuales.
Despliegue	docker compose up --build levanta db, backend y frontend de forma reproducible.
Documentación	La memoria, manual técnico y manual de usuario explican instalación, uso y evaluación.

7.3 Gestión de incidencias, cambios y registro

Las incidencias se registran con identificador, fecha, descripción, pasos para reproducir, severidad, responsable, solución aplicada y estado. El procedimiento consiste en reproducir el fallo, aislar capa afectada, revisar logs, proponer corrección, implementar en rama, probar y cerrar con evidencia. Los cambios en recursos o actividades se gestionan mediante solicitud de cambio: descripción, motivo, impacto en tiempo/coste, riesgos, decisión y actualización de planificación. Este proceso evita modificaciones improvisadas que comprometan requisitos ya validados.

Campo de incidencia	Ejemplo
ID	INC-004
Descripción	Al cambiar a READ no se guarda endDate.

Campo de incidencia	Ejemplo
Severidad	Alta, afecta a estadísticas anuales.
Solución	Actualizar BookService para asignar fecha de fin y añadir prueba manual.
Evidencia	Captura del dashboard y respuesta JSON del endpoint.

7.4 Documentación para la evaluación y participación de usuarios

La documentación de evaluación incluye plan de pruebas, checklist de requisitos, matriz de trazabilidad, registro de incidencias, acta de validación, manual de usuario y guía técnica de despliegue. La participación de usuarios o clientes se realiza mediante pruebas guiadas: registro, añadir libro, modificar estado, aplicar filtros, consultar estadísticas y cambiar objetivo anual. Tras la prueba se recoge feedback sobre claridad de interfaz, utilidad del dashboard, facilidad de uso y errores detectados. Las observaciones se clasifican como errores, mejoras de usabilidad o nuevas funcionalidades, priorizándolas según impacto y esfuerzo.

7.5 Matriz de trazabilidad requisitos-implementación

Req.	Descripción	Implementación	Evidencia
RF01	Registro e inicio de sesión	AuthService, UserRepository, JwtTokenProvider, SecurityConfig	POST /api/auth/register y /login
RF02	CRUD de libros	BookService, BookController, BookRepository	GET/POST/PUT/DELETE /api/books
RF03	Estados de lectura	ReadingStatus, endpoint de status, fechas automáticas	PUT /api/books/{id}/status
RF04	Filtros por texto, estado, autor, género y editorial	Consultas en servicio/repositorio y parámetros frontend	Pantalla Biblioteca
RF05	Dashboard y objetivo anual	DashboardService y endpoint goal	/api/dashboard/summary y /goal
RF06	Estadísticas mensuales y géneros	DashboardService/statistics y gráficos en Vue	/api/dashboard/statistics
RF07	Catálogo global	CatalogService y CatalogController	GET /api/catalog
RNF01	Seguridad y aislamiento	JWT, BCrypt, filtrado por usuario autenticado	Prueba cruzada con dos usuarios
RNF02	Despliegue reproducible	Dockerfiles y docker-compose.yml	docker compose up --build
RNF03	Persistencia	MySQL con volumen db_data	Reinicio sin pérdida de datos

7.6 Cumplimiento del pliego de condiciones

El sistema para garantizar el cumplimiento del pliego se basa en una lista de verificación vinculada a la matriz de trazabilidad. Cada requisito debe tener una implementación identificada, una prueba asociada y una evidencia. No se considera entregable una funcionalidad que solo esté descrita en la memoria si no puede demostrarse en la aplicación. Antes de la entrega se ejecuta una revisión final: arranque de contenedores,

login, operaciones de libros, estadísticas, objetivo anual, catálogo, aislamiento entre usuarios, revisión de documentación y verificación de que no existen contraseñas reales ni secretos en el repositorio.

7.7 Plan de pruebas sintético

Caso	Pasos	Resultado esperado
CP01 Registro	Crear usuario con email único y contraseña válida	Usuario creado y contraseña no visible en claro.
CP02 Login	Enviar credenciales correctas	JWT recibido y acceso a rutas privadas.
CP03 CRUD	Crear, editar, listar y eliminar libro	Los cambios se reflejan en biblioteca y base de datos.
CP04 Estados	Pasar de PENDING a IN_PROGRESS y READ	Se actualizan startDate y endDate según corresponda.
CP05 Filtros	Filtrar por autor, género, editorial, texto y estado	La lista muestra solo coincidencias.
CP06 Dashboard	Añadir libros leídos del año	Resumen y progreso anual se actualiza.
CP07 Aislamiento	Usuario A intenta acceder a libro de B	Acceso denegado o recurso no disponible.
CP08 Despliegue	Reconstruir contenedores desde cero	Los servicios arrancan en orden y la API responde.

8. Conclusiones y referencias

BookTrack constituye un proyecto completo y coherente con los resultados de aprendizaje del ciclo de Desarrollo de Aplicaciones Web. Integra análisis del sector, diseño técnico, planificación, ejecución, evaluación, seguridad, persistencia y despliegue. La solución propuesta es viable como prototipo académico y dispone de una base sólida para evolucionar hacia un producto SaaS: arquitectura por capas, contenedores, autenticación JWT, base de datos relacional, dashboard y trazabilidad de requisitos. La principal mejora futura sería endurecer el entorno de producción mediante HTTPS, gestión externa de secretos, migraciones de base de datos versionadas, CI/CD, pruebas automatizadas y monitorización.

Referencias normativas y programas consultados

- BOE: Ley 31/1995, de Prevención de Riesgos Laborales.
- BOE: Real Decreto Legislativo 2/2015, texto refundido del Estatuto de los Trabajadores.
- Agencia Tributaria: manuales de obligaciones fiscales para empresarios, profesionales y sociedades.
- BOE: Reglamento (UE) 2016/679, Reglamento General de Protección de Datos, y Ley Orgánica 3/2018, de Protección de Datos Personales y garantía de los derechos digitales.
- BOE: Real Decreto 311/2022, Esquema Nacional de Seguridad, como referencia de buenas prácticas para sistemas de información del sector público.
- Acelera pyme / Red.es: Kit Digital y servicios de digitalización para pymes y autónomos.
- CDTI: programas de ayudas a proyectos de I+D e innovación empresarial.

9. Comandos Docker

Desde la carpeta booktrack: para arrancar con Docker Compose →
docker compose up --build -d

- Frontend: <http://localhost:3000>
- Backend: <http://localhost:8081>

Para detener: docker compose down

Otras anotaciones están en el readme de la carpeta: booktrack